Estruturas de Dados e Notação Big-O no Projeto Collision — Entrega 2

1. Introdução e escopo

Este documento identifica, no código da Entrega 2 do projeto Collision, onde aparecem vetores, listas, percorrimento de listas, acesso por chave (tabela hash) e a notação Big-O associada a cada trecho. Para cada trecho são apresentados: a localização, o código comentado, a função no sistema, a análise de complexidade e a boa prática de estrutura de dados que ele ilustra.

Convenções de notação:

- **n** = itens de uma lista;
- **m** = caracteres de um texto;
- **k** = chaves de um objeto;
- **L** = tamanho de um texto em um campo;
- **q** = tamanho da query string da URL;
- **f** = arquivos de uma pasta.

2. Mapa do projeto

Marcadores: 📋 lista/vetor · 🔁 percorrimento · 🔑 acesso por chave (dicionário/Set).

text

```
Collision/
├── Backend/
│   ├── src/
│   │   ├── repositorios/acompanhamentos.js  📋🔁🔑 where, args, filtros
│   │   ├── servicos/cadastros.js            🔑 Dicionário de configuração
│   │   ├── validadores.js                   🔁🔑 includes, Object.keys, Set
│   │   ├── documentos/arquivos.js           🔁 CSV, XLSX, PDF
│   │   └── app.js                           📋🔁 Vetor de rotas
│   └── sql/migracoes/001_estrutura.sql      🔑 Índices B-tree
│
└── Frontend/
    └── src/
        ├── data/                            ★ Listas e dicionários de dados
        │   ├── dadosAdmin.js                📋 revisoes, bimestres, atalhos
        │   └── dadosDashboard.js            📋🔑 menu[] e indicadores[]
        ├── componentes/dashboard/           🔁 .map em listas
        └── utilitarios/nomes.js             🔁 split → map → join
```

3. Boas práticas de estrutura de dados aplicadas

1. **Separar dados de interface.** No Frontend, os dados ficam em `src/data/` e os componentes apenas os consomem. No Backend, a lógica de banco fica em `repositorios/`.
2. **Escolher a estrutura pelo padrão de acesso.** Sequência pede vetor; busca por identificador pede dicionário ou Set (acesso O(1) em média).
3. **Indexar em vez de percorrer.** O uso de dicionários no Frontend evita cadeias de `if/else` e buscas lineares O(n).
4. **Limitar o tamanho da entrada.** Paginação de até 100 itens e exportação de até 10.000 registros transformam custos grandes em limitados.
5. **Delegar buscas ao banco.** Chaves primárias e restrições UNIQUE permitem busca O(log n).
6. **Acumular em lista e juntar uma vez.** Evita cópias repetidas de strings/bytes.
7. **Lista branca em dicionário.** Nomes de tabela/coluna vêm de uma estrutura fixa, garantindo segurança contra SQL Injection.
8. **Chaves estáveis em listas renderizadas.** No React, a `key` única permite atualizar apenas o que mudou na tela.

4. Vetores e listas

4.1 Backend: `src/repositorios/acompanhamentos.js` — estados e filtros

javascript

```
/** Estados do fluxo de revisão de um acompanhamento. Vetor de strings. */
export const estados = [
  "RASCUNHO", "ENVIADO_PARA_REVISAO", "EM_REVISAO",
  "DEVOLVIDO", "PUBLICADO", "CANCELADO",
];

export function filtrosAcompanhamento(query, ator) {
  const acesso = acessoAcompanhamento(ator),
    where = [acesso.sql],        // lista de condições SQL (cresce com push)
    args = [...acesso.args];     // lista dos valores dos "?" (cópia: não altera o original)

  // Dicionário: nome do parâmetro na URL -> coluna do banco.
  const mapa = {
    alunoId: "id_aluno", turmaId: "id_turma", disciplinaId: "id_disciplina",
    professorId: "id_professor", bimestreId: "id_bimestre",
    anoLetivo: "ano_letivo", bimestre: "bimestre_numero",
  };

  // Percorre as 7 entradas; só cria o filtro se o parâmetro foi enviado.
  for (const [api, coluna] of Object.entries(mapa))
    if (query[api] !== undefined) {
      where.push("a." + coluna + "=?");        // push: O(1) amortizado
      args.push(v.inteiro(query[api], api));   // valida antes de usar
    }

  return { sql: where.join(" AND "), args };
}
```

Função: Monta a cláusula WHERE e a lista de valores para as listagens de acompanhamentos.

Big-O: O(k + w), praticamente constante (k=7, w=tamanho do texto). Cada push é O(1) amortizado.

Boa prática: Valores ficam em `args` (separados do SQL), nunca concatenados na string, mantendo a segurança e a organização.

4.2 Frontend: `src/data/dadosAdmin.js` — listas de dados do painel

jsx

```
/**
 * Fila de revisão exibida no painel do administrador.
 * Estrutura: vetor de vetores (lista de registros com posições fixas).
 * Formato: [iniciais, nome, turma, detalhe, data, estado, cor]
 */
export const revisoes = [
  ["ML", "Monkey D. Luffy", "Matematica . 8º A", "enviado por Prof. Chico", "Hoje, 10:32", "Pendent
e", "laranja"],
  ["RZ", "Roronoa Zoro", "Geografia . 8º A", "Enviado por Prof. AArlong", "Ontem, 14:20", "Pendente",
"laranja"],
];
```

Função: Alimenta a tela de fila de revisão do administrador.

Big-O: Criar o vetor é O(n) uma vez. Ler uma posição (`revisoes[0]`) é O(1). Percorrer é O(n).

Boa prática: Isolar dados da interface permite trocar o mock por API sem alterar a tela.

---

5. Percorrimento de listas e Big-O

5.1 Backend: `dtoAcompanhamentos` — junção de dados com paginação

javascript

```
export async function dtoAcompanhamentos(db, registros, ator) {
  if (!registros.length) return [];
  const ids = registros.map((r) => r.id_acompanhamento),     // O(n)
    marks = ids.map(() => "?").join(",");                    // O(n)
```

```
  // Busca otimizada: uma única consulta IN (...) para todos os registros.
  const tags = await linhas(db, `SELECT ... WHERE atg.id_acompanhamento IN (${marks})`, ids);
  const retornos = await linhas(db, `SELECT ... WHERE id_acompanhamento IN (${marks}) ...`, [...]);

  return registros.map((r) => ({          // laço externo: n registros
    id: r.id_acompanhamento,
    tags: tags
      .filter((t) => t.id_acompanhamento === r.id_acompanhamento)   // laço interno
      .map((t) => ({ id: t.id_tag, nome: t.nome, cor: t.cor })),
    // ...
  }));
}
```

Função: Junta tags e retornos a cada acompanhamento para enviar à API.

Big-O: O custo é proporcional a n · (T + R). Como o sistema aplica paginação de até 100 registros para listagens e limita a exportação a 10.000, o processamento mantém-se sob controle e performático.

Boa prática: O uso de uma única consulta `IN (...)` evita o problema de N+1 consultas no banco de dados, que seria muito mais lento.

5.2 Frontend: `ConteudoAdmin.jsx` — renderização de listas

jsx

```
{revisoes.map(([iniciais, nome, turma, detalhe, data, estado, cor]) => (
  <button className="dash-linha" key={nome} onClick={() => onNavegar("Revisão")}>
    <span className="dash-iniciais">{iniciais}</span>
    <small className="dash-data">{data}</small>
    <span className={`dash-status dash-status--${cor}`}>{estado}</span>
  </button>
))}
```

Função: Desenha a fila de revisão na tela do administrador.

Big-O: O(n). Cada `.map` visita os itens uma vez, e os laços rodam em sequência.

Boa prática: A `key` única mantém a atualização do React eficiente, e a desestruturação do vetor deixa o código limpo e legível.

5.3 Backend: `scripts/check.js` — recursão (busca em profundidade)

javascript

```
async function visitar(pasta) {
  for (const entrada of await fs.readdir(pasta, { withFileTypes: true })) {
    const nome = path.join(pasta, entrada.name);
    if (entrada.isDirectory()) await visitar(nome);   // chamada recursiva
    else if (nome.endsWith(".js")) { /* verifica sintaxe */ }
  }
}
```

Função: Script de qualidade que checa a sintaxe de todos os arquivos `.js`.

Big-O: O(N) no número de entradas visitadas. Espaço extra: **O(d)** (profundidade da árvore de pastas na pilha de chamadas).

Boa prática: Estruturas hierárquicas (pastas) são percorridas de forma natural e eficiente com recursão.

5.4 Frontend: `utilitarios/nomes.js` — manipulação de strings

jsx

```
export function obterIniciais(nome) {
  return nome
    .trim()                          // O(m)
    .split(/\s+/)                    // O(m)
    .slice(0, 2)                     // O(1)
    .map((parte) => parte.charAt(0))  // O(1), pois restam 2 itens
    .join("")                        // O(1)
    .toUpperCase();                  // O(1)
}
```

Função: Gera as iniciais do avatar (ex: "Lucas Kazuo" → "LK").

Big-O: O(m) (tamanho do texto), dominado pelas operações de `trim` e `split`.

Boa prática: O uso do `slice(0, 2)` antes do `map` otimiza a execução, pois processa apenas o necessário.

---

6. Acesso por chave e tabela hash (implícita)

6.1 Backend: `servicos/cadastros.js` — dicionário de configuração

javascript

```
export const cadastros = {
  areas: {
    tabela: "AREA_DISCIPLINA", pk: "id_area",
    campos: {
      nome: ["nome", (x) => v.texto(x, "nome")],
      descricao: ["descricao", (x) => v.texto(x, "descricao", 1000, false)],
    },
  },
  // disciplinas, turmas, alunos, tags: mesma forma
};

function dto(tipo, r) {
  const c = cadastros[tipo],                              // O(1): busca por chave
    saida = { id: r[c.pk], ativo: !!r.ativo };
  for (const [api, [sql]] of Object.entries(c.campos))    // O(k): campos do cadastro
    saida[api] = r[sql];
  return saida;
}
```

Função: Um único conjunto de funções atende o CRUD de áreas, disciplinas, turmas, alunos e tags.

Big-O: `cadastros[tipo]` é **O(1)** em média.

Boa prática: O dicionário funciona como lista branca (tipo só pode ser chave existente), eliminando a duplicação de código e garantindo segurança.

6.2 Frontend: `Dashboard.jsx` — dicionário de componentes

jsx

```
const conteudos = {
  professor: ConteudoProfessor,
  responsavel: ConteudoResponsavel,
  admin: ConteudoAdmin,
};

// ...dentro do componente:
const Conteudo = conteudos[usuario.perfil]; // O(1)
```

Função: Decide qual painel desenhar na tela, evitando cadeias de `if/else`.

Big-O: O(1) em média.

Boa prática: Dicionários no lugar de condicionais mantêm a escolha em tempo constante e tornam a inclusão de um novo perfil uma alteração de apenas uma linha.

6.3 Backend: `Set` para unicidade

javascript

```
// validadores.js
const resultado = valor.map((x) => inteiro(x, campo));   // O(n)
if (new Set(resultado).size !== resultado.length)        // Set: O(n)
  invalido(campo, "Não repita identificadores.");
```

Função: Detecta identificadores duplicados em listas de até 30 itens.

Big-O: O(n), substituindo a comparação par a par (que seria $O(n^2)$).

Boa prática: O `Set` é a estrutura ideal para detectar duplicidade de forma rápida e eficiente.

7. Quadro-resumo de Big-O

Local	Estrutura	Operação	Big-O
Backend `acompanhamentos.js`	Vetor, dicionário	Montar filtros	O(k + w)
Backend `dtoAcompanhamentos`	`.map` + `.filter`	Juntar tags/retornos	O(n · (T + R))
Backend `validadores.js` (objeto)	`Object.keys` + `includes`	Lista branca de campos	O(k · p)
Backend `validadores.js` (ids)	`.map` + `Set`	Detectar repetidos	O(n)
Backend `arquivos.js` (CSV/XLSX)	Matriz, `.map`	Gerar linhas	O(n · c) ≈ O(n)
Backend `migrar.js`	`.sort()`	Ordenar migrações	O(f log f)
Backend `check.js`	Recursão	Percorrer pastas	O(N), espaço O(d)
Backend `auth.js`	SHA-256, bcrypt	Hash de token/senha	O(m) / 2^{12}
Backend `sql/001_estrutura.sql`	Índices B+tree	Busca por PK/UNIQUE	O(log n)
Frontend `dadosAdmin.js`	Vetor	Ler posição	O(1)
Frontend `ConteudoAdmin.jsx`	`.map`	Renderizar listas	O(n)
Frontend `Dashboard.jsx`	Dicionário	Escolher componente	O(1)
Frontend `nomes.js`	Split/map/join	Gerar iniciais	O(m)
Frontend `login.jsx`	`Object.keys`	Validar campos	O(k)

8. Conclusão

No projeto Collision, as estruturas de dados foram aplicadas de forma estratégica tanto no Backend quanto no Frontend. No Backend, destacam-se o uso de vetores para montagem dinâmica de filtros SQL, dicionários para configuração de cadastros e a otimização de consultas em lote (evitando N+1) na exportação de relatórios. No Frontend, o uso de dicionários (tabelas hash implícitas) para roteamento de componentes e listas para renderização dinâmica com `.map` garante uma interface fluida e de fácil manutenção.

A aplicação da notação Big-O permitiu ao grupo compreender e controlar a complexidade dos algoritmos (desde operações O(1) até O(n log n) em ordenações e O(log n) em índices de banco), garantindo que o sistema seja performático mesmo com o crescimento do volume de dados, respeitando os limites de paginação e exportação definidos na arquitetura. O código reflete, portanto, o domínio prático e teórico das estruturas de dados estudadas.